

Yes. This is very common in **enterprise Spring Boot projects**, especially microservices.

The key point is:

VM arguments are not required by Spring Boot itself. They are required when the application needs JVM-level settings or when the project expects configuration to be supplied externally.

There are usually **three different things** people call "configuration", and they are easy to confuse.

1. Spring Boot application configuration

Example:

```
properties

server.port=8081

spring.datasource.url=jdbc:postgresql://localhost:5432/payment_db
spring.datasource.username=postgres
spring.datasource.password=password
```

These are **application properties**.

Spring Boot reads them from places such as:

```
application.properties
application.yml
environment variables
command-line arguments
external configuration files
```

For example:

```
Bash

java -jar payment-service.jar --server.port=8085
```

This is a **Spring Boot command-line argument**, not technically a JVM argument.

2. JVM arguments / VM arguments

VM arguments configure the **JVM itself**, rather than Spring Boot.

For example:

```
-Xms512m  
-Xmx2g  
-Dspring.profiles.active=dev  
-Dserver.port=8081  
-Dfile.encoding=UTF-8
```

Notice the `-D`.

For example:

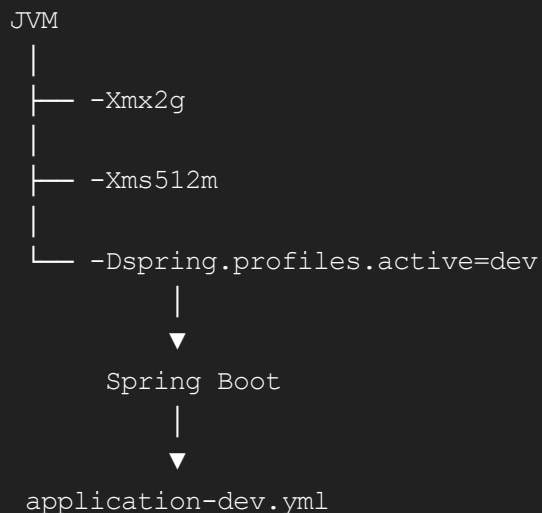
```
-Dspring.profiles.active=dev
```

means:

Create a Java system property called `spring.profiles.active` with value `dev`.

Spring Boot can then read that system property.

So:



3. Why would a project need VM arguments?

There are several common reasons.

Reason 1 — Selecting Spring profile

This is probably the **most common reason** you'll encounter.

Suppose your project has:

```
src/main/resources/
|
├─ application.yml
├─ application-dev.yml
├─ application-test.yml
└─ application-prod.yml
```

`application-dev.yml`:

YAML

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/payment_dev
```

Production:

YAML

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://production-db:5432/payment
```

You need to tell Spring which profile to activate.

You might configure:

```
-Dspring.profiles.active=dev
```

Then Spring loads:

```
application.yml
+
application-dev.yml
```

If you use IntelliJ:

```
Run → Edit Configurations → VM options
```

and enter:

```
-Dspring.profiles.active=dev
```

4. Why not simply put it in application.yml?

You could do:

YAML

```
spring:
  profiles:
    active: dev
```

But this is often undesirable.

Why?

Because you don't want the application itself to decide:

"I am running in development."

The deployment environment should decide that.

For example:

Developer laptop

↓

profile = dev

QA server

↓

profile = qa

Production server

↓

profile = prod

The same JAR can then be deployed everywhere.

payment-service.jar

|

├─ DEV

├─ QA

└─ PROD

Only the external configuration changes.

This is an important **12-factor application** principle.

5. Reason 2 — Environment variables / secrets

Enterprise applications shouldn't have:

YAML

```
spring:
  datasource:
    password: MySecretPassword
```

committed to Git.

Instead:

```
DB_PASSWORD=*****
```

and configure Spring to read it:

YAML

```
spring:
  datasource:
    password: ${DB_PASSWORD}
```

Then your run configuration might contain:

Environment variables:

```
DB_PASSWORD=*****
DB_USERNAME=postgres
```

Sometimes you'll see VM arguments such as:

```
-Ddb.password=*****
```

and then:

YAML

```
spring:
  datasource:
    password: ${db.password}
```

6. Reason 3 — JVM memory

This is a completely different use of VM arguments.

Suppose your application processes large amounts of data.

You may see:

```
-Xms512m  
-Xmx2048m
```

Meaning approximately:

```
-Xms512m  
  ↓  
Initial JVM heap = 512 MB  
  
-Xmx2048m  
  ↓  
Maximum JVM heap = 2 GB
```

This controls the Java heap.

It doesn't configure Spring Boot.

It's configuring the **JVM running Spring Boot**.

7. Reason 4 — Java modules / reflection

Some enterprise applications may have VM arguments like:

```
--add-opens java.base/java.lang=ALL-UNNAMED
```

or:

```
--add-opens java.base/java.util=ALL-UNNAMED
```

These are generally related to Java's module system and access restrictions.

You may encounter these with:

- older libraries
- Hibernate
- bytecode instrumentation

- mocking frameworks
- reflection-heavy frameworks

For example, after upgrading Java, an old dependency may fail because it tries to access an internal JDK class.

A VM argument can sometimes be used as a compatibility workaround.

8. Reason 5 — Remote debugging

This is another common one.

You might see:

```
-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=:5005
```

This allows a debugger to connect to the JVM.

For example:

```
IntelliJ
  |
  | Debug connection
  ▼
Spring Boot JVM
  |
  ▼
Port 5005
```

This is useful when debugging a service running inside Docker or another environment.

9. Reason 6 — Java system properties

You may see:

```
-Dsome.property=value
```

For example:

```
-Dspring.config.location=/opt/config/application.yml
```

This tells Spring Boot to look for configuration somewhere outside the JAR.

Another example:

```
-Dlogging.config=/opt/config/logback.xml
```

This tells the application where to find its logging configuration.

10. Why does IntelliJ ask for "Program arguments" and "VM options"?

This is an important distinction.

Suppose you have:

VM options

```
-Dspring.profiles.active=dev  
-Xmx2g
```

These go to the **JVM**.

Conceptually:

Bash

```
java -Xmx2g -Dspring.profiles.active=dev ...
```

Program arguments

```
--server.port=8085
```

These go to your **Spring Boot application's** `main()` **method**.

Conceptually:

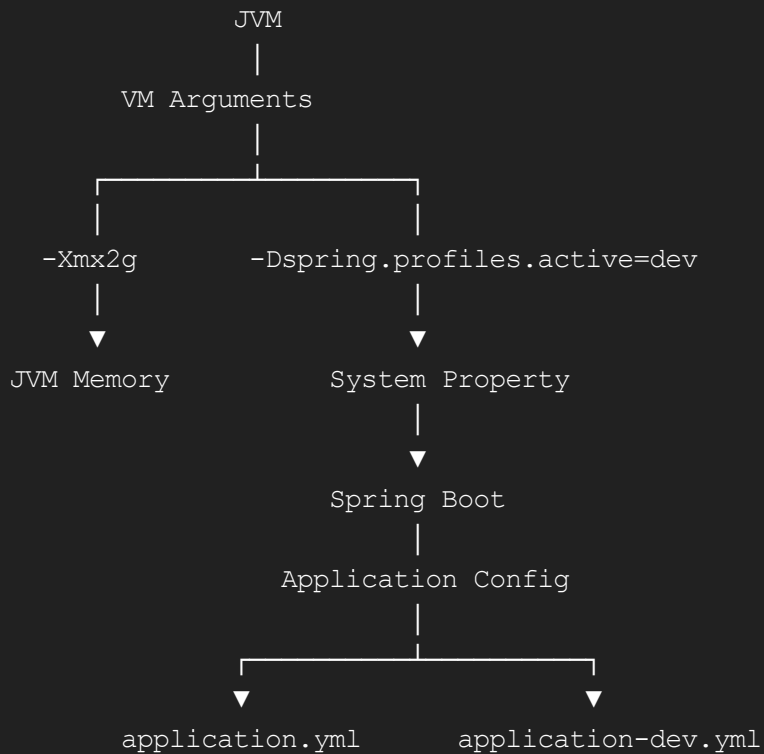
Bash

```
java ... com.example.PaymentApplication --server.port=8085
```

Spring Boot then parses the argument.

11. The three levels you should remember

This is the easiest way to understand it:



And separately:

Program Arguments

```
--server.port=8085  
--spring.profiles.active=dev
```

These are passed to the Spring Boot application.

12. Why enterprise projects have a lot of configuration

This becomes particularly important in **microservices**.

Imagine your Payment Service needs:

```
Database  
Redis  
Kafka  
User Service  
Wallet Service  
Fraud Service  
External Payment Gateway  
JWT  
Monitoring
```

Your local environment might be:

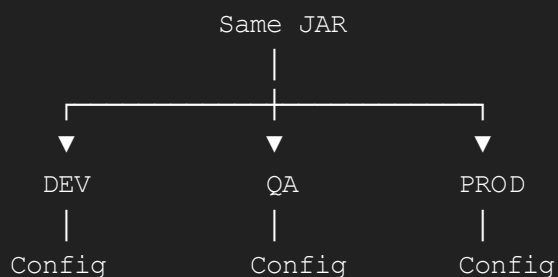
```
PostgreSQL → localhost:5432
Redis      → localhost:6379
Kafka      → localhost:9092
User API   → localhost:8081
Wallet API → localhost:8082
Fraud API  → localhost:8083
```

But production might be:

```
PostgreSQL → payment-db.internal
Redis      → redis.internal
Kafka      → kafka.internal
User API   → http://user-service:8080
Wallet API → http://wallet-service:8080
Fraud API  → http://fraud-service:8080
```

You don't want to modify Java code for every environment.

Instead:



That's why you'll often see a project with **lots of configuration files and run configurations**.

13. Example from a microservices project

Imagine:

```
YAML

# application.yml

services:
  user:
    url: ${USER_SERVICE_URL}

  wallet:
```

```
url: ${WALLET_SERVICE_URL}
```

```
fraud:
```

```
url: ${FRAUD_SERVICE_URL}
```

Development:

```
USER_SERVICE_URL=http://localhost:8081  
WALLET_SERVICE_URL=http://localhost:8082  
FRAUD_SERVICE_URL=http://localhost:8083
```

Docker:

```
USER_SERVICE_URL=http://user-service:8080  
WALLET_SERVICE_URL=http://wallet-service:8080  
FRAUD_SERVICE_URL=http://fraud-service:8080
```

Kubernetes:

```
USER_SERVICE_URL=http://user-service  
WALLET_SERVICE_URL=http://wallet-service  
FRAUD_SERVICE_URL=http://fraud-service
```

Same application code. Different configuration.

That's one of the major reasons configuration becomes important in microservices.

14. Why your project may fail without the configuration

Suppose you clone a GitHub Spring Boot project and simply run:

Bash

```
mvn spring-boot:run
```

and get:

```
Failed to configure a DataSource
```

or:

```
Could not resolve placeholder 'DB_URL'
```

or:

```
Connection refused: localhost:9092
```

or:

```
No active profile set
```

It doesn't necessarily mean the code is broken.

The project may expect:

```
Environment variables
    +
VM arguments
    +
Docker containers
    +
External configuration
    +
Kafka
    +
PostgreSQL
    +
Redis
```

to already be available.

15. A practical example

Suppose you clone:

```
payment-service
```

README says:

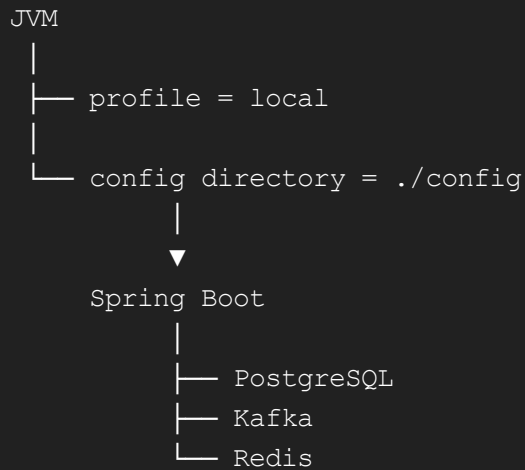
```
VM Options:

-Dspring.profiles.active=local
-Dspring.config.additional-location=./config/
```

And environment variables:

```
DB_URL
DB_USERNAME
DB_PASSWORD
```

When you start the application:



So the configuration is effectively telling Spring:

"Run using the local environment and load the configuration from this location."

The most important distinction

When you see something like this in a Spring Boot project:

```
-Dspring.profiles.active=dev
```

think:

"Spring environment selection."

When you see:

```
-Xmx2g
```

think:

"JVM memory."

When you see:

```
--server.port=8081
```

think:

"Spring Boot application argument."

When you see:

```
DB_PASSWORD=xxxx
```

think:

"Environment variable."

And when you see:

```
--add-opens ...
```

think:

"JVM/module compatibility."

This distinction will make the configuration of the larger microservices project we're discussing much easier to understand.